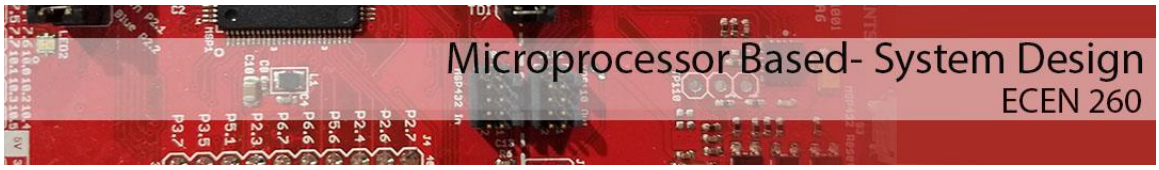


ECEN260 – Laboratory # 8

Analog to Digital Conversion on the MSP432

Objectives

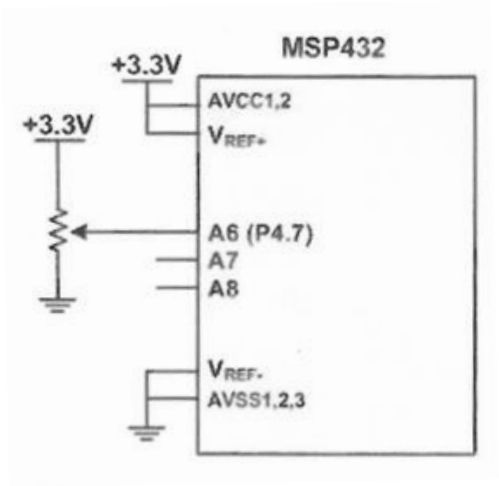
1) Learn how to configure and use the ADC14 in the MSP432.
2) Use the ADC14 to read an analog voltage, convert the result to digital, and output the result.

Overview

The MSP432 has one ADC with 14-bit resolution. It can support up to 24 ADC channels. This laboratory exercise will configure the ADC14 to use input channel 6. It will convert an analog input voltage and display its digital equivalent using the LEDs.

See section 8.4 in the textbook (Programmable Microcontrollers: Applications on the MSP432 LaunchPad) and section 22.3 of the Technical Reference Manual for more details about the ADC14 registers that will be used.

Connections



Note: The reference voltages are connected internally via control registers. The 3 pins of the potentiometer are the only things you need to connect.

This material is the property of the university. It is provided to students in ECEN 260 during Spring 2022. This material may not be shared with others nor posted online. Sharing or posting this material is academic dishonesty. Use of this material by students in other semesters is academic dishonesty.

Program Requirements

Connect a 10k potentiometer to analog input channel 6 of the ADC14, which is P4.7 (see previous figure). Rotating the wiper of the potentiometer will change the input voltage from 0 to 3.3V. ADC14 will be configured for 12 bits of resolution, which will use bits 0 through 11.

Step 1:

Connect Bits 8, 9, and 10 of the ADC to the tri-color LED2 on P2.0, P2.1, and P2.2 of the MSP432. Thus, three bits of the resulting digital conversion (8, 9, and 10) will be displayed on the tri-color LED. Since the highest bit is not used, the colors will cycle through two times.

Step 2:

Add LED1 (P1.0) as the output for the highest significant bit (bit 11). This will cause the full cycle of output lights (P1.0 and P2.2, P2.1, and P2.0) to represent the highest 4 significant bits. This extended cycle should only cycle one time.

Step 3:

Add an interrupt on SW1 (P1.1). Pushing this button should shift which bits get displayed to double the cycle length. For example, pushing the button one time will shift the four output lights to display bits 7, 8, 9, and 10 instead of the original 8, 9, 10, and 11, resulting in 2 full cycles. Pushing the button again will shift the four output lights to display bits 6, 7, 8, and 9, resulting in 4 full cycles. Pushing the button yet again will shift the four output lights to display bits 5, 6, 7, and 8, resulting in 8 full cycles.

Step 4:

Have every 5th button push revert to the original highest four bits, and then continue the rotation for subsequent pushes. Thus, pushing the button will rotate between cycles of 2^0=1, 2^1=2, 2^2=4, 2^3=8, 2^4=16, then back to 2^0=1, 2^1=2, 2^2=4, 2^3=8, 2^4=16, then back to 2^0=1, 2^1=2, ... etc.

(You only need to submit the final code after all 4 steps.)

This material is the property of the university. It is provided to students in ECEN 260 during Spring 2022. This material may not be shared with others nor posted online. Sharing or posting this material is academic dishonesty. Use of this material by students in other semesters is academic dishonesty.

ADC14 Registers

Only the ADC14 registers that are needed for this laboratory will be described.

CTL0 (Control Word 0)

Bit settings in Control Word 0 for this laboratory exercise:

Bits 31:30 (ADC14PDIV)	= 00 (predivide by 1)
Bits 29:27 (ADC14SHSx)	= 000 (ADC14SC bit)
Bit 26 (ADC14SHP)	= 1 (use sampling timer)
Bit 25 (ADC14ISSH)	= 0 (input signal not inverted)
Bits 24:22 (ADC14DIVx)	= 000 (divide by 1)
Bits 21:19 (ADC14SSELx)	= 001 (use the system clock: SYSCLK)
Bits 18:17 (ADC14CONSEQx)	= 00 (single channel, single conversion)
Bit 16 (ADC14BUSY)	= 0 (no operation is active)
Bits 15:12 (ADC14SHT1x)	= 0000 (sample period for mem regs 8 to 23 = 4)
Bits 11:8 (ADC14SHT0x)	= 0011 (sample period for mem regs 0 to 7 = 32)
Bit 7 (ADC14MSC)	= 0 (rising edge to trigger sample-and-convert)
Bits 6:5 (Reserved)	= (always 00)
Bit 4 (ADC14ON)	= 0 (ADC14 off)
	= 1 (ADC14 on)
Bits 3:2 (Reserved)	= (always 00)
Bit 1 (ADC14ENC)	= 0 (disabled - when configuring)
	= 1 (enabled - when operating)
Bit 0 (ADC14SC)	= 0 (no-sample-and-conversion-start)

CTL1 (Control Word 1)

The bit fields in this register that affect us for this laboratory are bits 20:16 and bits 5:4.

Bits 20:16 designate the memory register that will be used. In our case, we will use memory register 5. The binary number to load into these bits of the CTL1 for this laboratory is 00101b.

Bits 5:4 determine the resolution that will be used. The binary number to load into these bits for a resolution of 12 is 10b

This material is the property of the university. It is provided to students in ECEN 260 during Spring 2022. This material may not be shared with others nor posted online. Sharing or posting this material is academic dishonesty. Use of this material by students in other semesters is academic dishonesty.

MCTL0

The channel to be used is selected by the MCTL0. In our case, we will use MCTL[5], for memory register 5 and load bits 4:0 with 00110b for channel 6. Channel 6 is pin P4.7 on the LaunchPad.

GPIO Control Registers

PxSEL0 and PxSEL1 define MSP432 I/O pins as digital I/O, or as specialized peripheral pins. Upon MSP432 power-up or reset, all pins of Port 1 through Port 10 are configured as digital I/O (PxSEL0 = 00h and PxSEL1 = 00h). For this lab, you will configure them explicitly.

Configure Port 1 and Port 2 as simple digital I/0 because these ports will drive the LEDs or read digital input. Port 4, however, needs to be configured to work with the ADC14. To configure a pin as analog input, the corresponding bits in the SEL0 and SEL1 control registers should be set.

This material is the property of the university. It is provided to students in ECEN 260 during Spring 2022. This material may not be shared with others nor posted online. Sharing or posting this material is academic dishonesty. Use of this material by students in other semesters is academic dishonesty.

Laboratory Report Requirements (50 pts)

Note: The only parts of your Lab Report that should be copied from these lab instructions are the Objectives. Everything else should be in your own words.

Your report should include the following information:

- **Title Page:** Lab Title / Course / Your Name / Date
- **Section 1: Lab Overview (10 pts):**
  - *1.1 Objectives (1 pt):* See beginning of instructions.
  - *1.2 Explanation (2 pt):* Describe the lab in more detail.
  - *1.3 Specifications, Usage, and Limitations* (3pts): Specifications are a precise description of a working product/solution. Include the intended usage (operation procedures). Also include the limitations–what might someone reasonably expect the product to do but it doesn't?
  - *1.4 Logic Flow of the Code* (2pts): Describe the structure/flow of the code.
  - *1.5 Circuit Descriptions* (2pts): Describe any circuits used in the lab. Also include a parts list.
- **Section 2: Schematics (3 pts):** Include switch, LEDS, and potentiometer. No need to include internal pull resistor nor the internal ADC reference voltage connections.
- **Section 3: Code (10 pts):** Well-commented code (just the final code after step 4).
- **Section 4: Test Plan (10 pts):** How you verified correct operation. Include testing steps and expected results.
- **Section 5: Test Results (10 pts)**
  - *5.1 Recorded Test Results* (5 pts)*:* Actual results of each test scenario.
  - *5.2 Evidence of Correct Operation* (5 pts)*:* For this lab, as evidence of correct operation, record a video demonstrating the working prototype. Your video does not need to include every test—just show correct functionality.
- **Section 6: Conclusions (7 pts):** Ponder on what you learned and accomplished. Then write three paragraphs:
  - (2 pts) One paragraph summarizing what you learned and accomplished as if you were teaching it to someone else.
  - (3 pts) A second paragraph that discusses how this project connects to things you already knew, with what we have learned in class, or with what you hope to do in the future.
  - (2 pts) A third paragraph with some initial ideas for a final project.

(There is no need to include the Code-with-Me nor Peer Review sections.)

This material is the property of the university. It is provided to students in ECEN 260 during Spring 2022. This material may not be shared with others nor posted online. Sharing or posting this material is academic dishonesty. Use of this material by students in other semesters is academic dishonesty.

Scaffolding Code:

```
#include "msp.h"

#define ADCIN BIT7 // Port 4.7

// global variable to adjust bit shifting
int ???

void main(void){
    WDT_A->CTL = WDT_A_CTL_PW | WDT_A_CTL_HOLD; // stop watchdog timer

    int result;

    // Configure LED2RGB as outputs for tri-color LEDs
    P?->???  ?= ????;   // Set LED2 as outputs
    P?->SEL0 ?= ????;   // Reset SEL0 & SEL1 to 0 for Digital I/O
    P?->SEL1 ?= ????;

    // Configure LED1 as output LED
    P?->???  ?= ????;   // Set LED1 as output
    P?->SEL0 ?= ????;   // Reset SEL0 & SEL1 to 0 for Digital I/O
    P?->SEL1 ?= ????;

    // Configure SW1 as input
    P?->???  ?= ????;   // Set SW1 as input
    P?->???  ?= ????;   // Enable pull resistor
    P?->???  ?= ????;   // Specify as pull-up resistor
    P?->SEL0 ?= ????;   // Reset SEL0 & SEL1 to 0 for Digital I/O
    P?->SEL1 ?= ????;

    // Pin-level Interrupt Enable for P1.1 (SW1)
    P1->??? ?= ????;    // pin-level interrupt enable
    P1->??? ?= ????;    // falling edge
    P1->??? ?= ????;    // clear flag

    // Port-level Interrupt Enable for Port 1 (for SW1)
    NVIC->ISER[?] |= ???;
    _enable_interrupts();

    // Configure ADC14

    // -> Configure P4.7 for A6 (analog input channel 6)
    P4->SEL1 |= ADCIN;
    P4->SEL0 |= ADCIN;
```

```
    // -> Configure CTL0
    ADC14->CTL0 = 0x00000010;   // Power on ADC and disable during config
    ADC14->CTL0 |= ????;        // Configure CTL0 according to instructions

    // -> Configure CTL1
    ADC14->CTL1 = ????;         // 12-bit resolution (see instructions)
    ADC14->CTL1 |= 0x00050000;  // Configure for memory register 5

    // -> Configure MCTL[5] (memory register 5) to receive input channel 6
    ADC14->MCTL[5] = 0x06;      // A6 input (P4.7), single ended, Vref = AVCC

    // Enable ADC14
    ADC14->CTL0 |= ????;        // Enable ADC14 after configuration

    // Repeatedly sample analog signal and display digital conversion
    while (1){

        // Perform Analog-to-Digital Conversion (ADC)
        ADC14->CTL0 |= 1;       // Start a conversion
        while (!ADC14->IFGR0);  // Wait until conversion is complete
        result = ADC14->MEM[5]; // Read conversion result

        // Get 3 bits of result for LED2's tri-color LED
        P2->OUT = result >> (????);  // shift bits

        // Get 1 bit of result for LED1 (1 digit higher than previous 3)
        P1->OUT = ????; // shift bits
    }
}

// The SW1 interrupt is handled as part of the Port 1 Interrupt Handler
void ???(void){

    // Check if SW1 interrupt flag is high
    if (????){

        ????;       // adjust global variable
        if (????){  // if shifted enough times
            ????;   // reset global variable
        }

        // Debounce switch

        // clear interrupt flag
    }
}
```

This material is the property of the university. It is provided to students in ECEN 260 during Spring 2022. This material may not be shared with others nor posted online. Sharing or posting this material is academic dishonesty. Use of this material by students in other semesters is academic dishonesty.